

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE NAME: INTERNET PROGRAMMING

COURSE CODE: CSA43

Name:K.mrudhula

Reg no: 192372290

COURSE OUTCOME COVERED

CO-1: Develop well-structured, standards-compliant web pages using HTML/XHTML and XML, and apply Internet protocols and HTTP communication mechanisms for web content delivery. (L2)

Assessment Tool: Application-Based Questions

Weightage: 40%

QUESTIONS: Total Marks: 30

1: Online Symposium Portal

A college is developing an online symposium registration portal. The following HTML structure and HTTP interaction is observed:

```
<form action="/register" method="POST">
  <input type="text" name="name">
  <input type="email" name="email">
  <button>Submit</button>
</form>
```

When a student submits the form, the browser sends an HTTP request to the server, and the server responds accordingly.

Questions:

1. Identify appropriate **HTML5 semantic elements** missing in the structure.
2. Modify the structure to make it **standards-compliant**.
3. Interpret the **HTTP request generated** during form submission.
4. Analyze the **HTTP response cycle** from server to browser.
5. Suggest improvements for **usability and accessibility**.

2: Cross-Browser Compatibility Issue

A company website shows inconsistent layout across browsers. The following HTML snippet is used:

```
<html>
  <head>
    <title>Company</title>
  </head>
  <body>
    <div>
      <h1>Welcome
      <p>About us
    </div>
  </body>
</html>
```

Questions:

1. Identify **improper nesting and missing closing tags**.
2. Analyze how different browsers may interpret this structure.
3. Identify **missing semantic elements** (e.g., header, section).
4. Provide a **corrected W3C-compliant HTML structure**.
5. Suggest techniques to ensure **cross-browser compatibility**.

3: Form Submission Failure

A web application fails to send form data to the server. The following configuration is observed:

```
<form action="/submit" method="GET">
  <input type="text" name="username">
  <input type="password" name="password">
  <button type="button">Login</button>
</form>
```

Questions:

1. Identify issues in the **form configuration**.
2. Analyze why the **HTTP request is not triggered**.
3. Interpret the role of **GET method in this scenario**.
4. Propose a **corrected implementation**.
5. Suggest improvements for **secure data transmission**.

Code of Conduct:

I k.mrudhula(192372290) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

mrudhula.k

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Concepts	25%	Demonstrates thorough, accurate understanding of concepts	Good understanding with minor gaps	Basic understanding with some misconceptions	Poor understanding; major misconceptions
Explanation	25%	Detailed, well explained answers with relevant examples	Clear explanation with adequate details	Limited explanation; lacks depth	Poor or unclear explanation
Application	20%	Effectively applies concepts with strong analytical ability	Applies concepts with minor errors	Limited application with weak analysis	No application or incorrect analysis
Organization	15%	Well-structured answers with logical flow and coherence	Organized with minor issues in flow	Basic structure, lacks clarity	Poor organization, incoherent answers
Accuracy	10%	Completely accurate and covers all required points	Mostly accurate with minor omissions	Partially correct with missing points	Incorrect answers with major gaps
Clarity	5%	Clear, neat, professional presentation	Understandable with minor issues	Limited clarity, readability issues	Poorly written, difficult to understand

1: Online Symposium Portal

A college is developing an online symposium registration portal. The following HTML structure and HTTP interaction is observed:

```
<form action="/register" method="POST">
  <input type="text" name="name">
  <input type="email" name="email">
  <button>Submit</button>
</form>
```

When a student submits the form, the browser sends an HTTP request to the server, and the server responds accordingly.

Questions:

1. Identify appropriate **HTML5 semantic elements** missing in the structure. 2. Modify the structure to make it **standards-compliant**.
3. Interpret the **HTTP request generated** during form submission.
4. Analyze the **HTTP response cycle** from server to browser.
5. Suggest improvements for **usability and accessibility**.

Solution:

1. Online Symposium Portal

1. Identify appropriate HTML5 semantic elements missing in the structure

The given code is missing the following HTML5 semantic elements:

- **<html>**
- **<head>**
- **<title>**
- **<body>**
- **<header>**
- **<main>**
- **<section>**
- **<label>** for input fields

2. Standards-Compliant HTML5 Structure

```
<!DOCTYPE html>
```

```
<html lang="en">
```

```
<head>
```

```
  <meta charset="UTF-8">
```

```
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
```

```
<title>Online Symposium Registration</title>
</head>

<body>

<header>
  <h1>Online Symposium Registration</h1>
</header>

<main>
<section>

<form action="/register" method="POST">

<label for="name">Name:</label>
<input type="text" id="name" name="name" required>

<br><br>

<label for="email">Email:</label>
<input type="email" id="email" name="email" required>

<br><br>

<button type="submit">Submit</button>

</form>

</section>
</main>

</body>
</html>
```

3. HTTP Request Generated During Form Submission

When the student clicks **Submit**, the browser sends an HTTP **POST** request to the server.

Example HTTP Request

POST /register HTTP/1.1

Host: www.college.edu

Content-Type: application/x-www-form-urlencoded

name=John&email=john@example.com

Explanation

- **POST** → Sends data securely to the server.
 - **/register** → URL where form data is sent.
 - **Host** → Server address.
 - **Content-Type** → Indicates form data format.
 - **Body** → Contains the student's name and email.
- ## 4. HTTP

Response Cycle from Server to Browser

1. Student fills the registration form.
2. Browser sends an HTTP POST request.
3. Server receives and validates the data.
4. Server stores the registration details in the database.
5. Server sends an HTTP response.
6. Browser displays the response page.

Example HTTP Response

HTTP/1.1 200 OK

Content-Type: text/html

```
<html>
<body>
<h2>Registration Successful</h2>
</body>
</html>
```

If there is an error, the server may return:

HTTP/1.1 400 Bad Request

or

HTTP/1.1 500 Internal Server Error

5. Improvements for Usability and Accessibility

- Use `<label>` for every input field.
- Add the `required` attribute for mandatory fields.
- Use meaningful headings (`<h1>`).
- Include `<header>`, `<main>`, and `<section>` semantic elements. •

Add placeholders (optional) to guide users.

- Ensure keyboard accessibility.
- Use proper color contrast for readability.
- Display validation error messages clearly.
- Add `autocomplete` attributes for faster form filling.
- Make the page responsive using the viewport meta tag

Result

The symposium registration page is redesigned using HTML5 semantic elements, follows web standards, correctly uses the HTTP POST request-response cycle, and provides improved usability and accessibility.

2: Cross-Browser Compatibility Issue

A company website shows inconsistent layout across browsers. The following HTML snippet is used:

```
<html>
  <head>
    <title>Company</title>
  </head>
  <body>
    <div>
      <h1>Welcome
      <p>About us
    </div>
  </body>
</html>
```

Questions:

1. Identify **improper nesting and missing closing tags**.
2. Analyze how different browsers may interpret this structure.
3. Identify **missing semantic elements** (e.g., header, section).
4. Provide a **corrected W3C-compliant HTML structure**.
5. Suggest techniques to ensure **cross-browser compatibility**.

1. Identify Improper Nesting and Missing Closing Tags

The given HTML has the following errors:

- Missing `<!DOCTYPE html>` declaration.
- Missing `<meta charset="UTF-8">`.
- Missing `<meta name="viewport">`.
- `<h1>` tag is not closed.
- `<p>` tag is not closed.
 - Uses `<div>` instead of semantic elements like `<header>` and `<section>`.

2. Analyze How Different Browsers May Interpret This Structure

Different browsers may automatically correct the invalid HTML in different ways.

- **Google Chrome:** Automatically closes missing tags and displays the page correctly.
- **Mozilla Firefox:** Repairs the HTML but layout may differ slightly.
- **Microsoft Edge:** Tries to correct the structure based on HTML5 parsing rules.
- **Safari:** Also auto-corrects the HTML, but rendering may vary.

Because the HTML is invalid, browsers may produce inconsistent layouts, affecting cross-browser compatibility.

3. Identify Missing Semantic Elements

The following semantic elements are missing:

- `<header>`
- `<main>`
- `<section>`

These improve page structure, readability, SEO, and accessibility.

4. Corrected W3C-Compliant HTML Structure


```
<!DOCTYPE html>

<html lang="en">

<head>

  <meta charset="UTF-8">

  <meta name="viewport" content="width=device-width, initial-scale=1.0">

  <title>Company</title>

</head>

<body>

<header>

  <h1>Welcome</h1>

</header>

<main>

<section>

  <p>About us</p>

</section>

</main>

</body>

</html>
```

5. Techniques to Ensure Cross-Browser Compatibility

- Use valid HTML5 syntax and proper closing tags.
- Validate code using the W3C HTML Validator.

- Use HTML5 semantic elements.
- Test the website in multiple browsers (Chrome, Firefox, Edge, Safari).
- Use CSS Reset or Normalize.css for consistent styling.
- Follow web standards and avoid deprecated tags.
- Use responsive design with the viewport meta tag.
- Test on different screen sizes and devices.

Result

The HTML code is corrected to follow W3C standards using proper nesting, semantic HTML5 elements, and valid syntax. This improves browser compatibility, accessibility, maintainability, and provides a consistent layout across different web browsers.

3: Form Submission Failure

A web application fails to send form data to the server. The following configuration is observed:

```
<form action="/submit" method="GET">
  <input type="text" name="username">
  <input type="password" name="password">
  <button type="button">Login</button>
</form>
```

Questions:

1. Identify issues in the **form configuration**.
2. Analyze why the **HTTP request is not triggered**.
3. Interpret the role of **GET method in this scenario**.
4. Propose a **corrected implementation**.
5. Suggest improvements for **secure data transmission**.

Solution:

3. Form Submission Failure

1. Identify Issues in the Form Configuration

The given form has the following issues:

- The button uses `type="button"` instead of `type="submit"`.
- The **GET** method is used to send login credentials, which is not secure.
- The input fields do not have the `required` attribute.

- Password is exposed in the URL when using the GET method.

2. Analyze Why the HTTP Request is Not Triggered

The HTTP request is not sent because:

- `<button type="button">` only displays a button.
- It does **not** submit the form automatically.
- Only `<button type="submit">` or JavaScript form submission will trigger an HTTP request.

3. Interpret the Role of GET Method in This Scenario

The **GET** method:

- Sends form data through the URL.
- Appends data as query parameters.
- Is suitable for searching and retrieving data.
- Is **not recommended** for login forms because usernames and passwords become visible in the browser address bar and may be stored in browser history.

Example URL:

`/submit?username=Rahul&password=12345`

4. Corrected Implementation

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Login Form</title>
</head>
<body>
<form action="/submit" method="POST">
<label for="username">Username:</label>
<input type="text" id="username" name="username" required>
<br><br>
<label for="password">Password:</label>
<input type="password" id="password" name="password" required>
<br><br>
<button type="submit">Login</button>
</form>
</body>
</html>
```

5. Improvements for Secure Data Transmission

- Use the POST method instead of GET for login forms.
- Use HTTPS to encrypt data during transmission.
- Add the **required** attribute for mandatory fields.
- Validate user input on both the client and server sides.
- Store passwords securely using hashing techniques on the server. • Implement authentication protections such as session management or tokens.
- Avoid exposing sensitive information in URLs.

Result

The login form is corrected by changing the button to **type="submit"**, using the POST method, and adding validation. Using HTTPS and server-side security measures ensures safe and reliable transmission of user credentials.